

AI 프롬프트 구조 설계

Persona · Context · Task로 결과를 바꾸는 법

AI 프롬프트 구조 설계

- “AI는 똑똑하지만 눈치가 없다”

- 지시하지 않으면 안 한다
- 애매하면 대충 답한다
- 잘못 물으면 잘못 답한다

☞ 핵심 메시지

결과 = 프롬프트 설계 능력

AI 프롬프트 구조 설계

✕ 나쁜 프롬프트

"마케팅 전략 알려줘"

👉 결과

- 너무 일반적
- 쓸 수 없음

AI 프롬프트 구조 설계

✓ 개선된 프롬프트

"스타트업을 위한 SNS 마케팅 전략을 3단계로 설명해줘. 예시 포함."

👉 결과

- 구체적
- 실행 가능

AI 프롬프트 구조 설계

- 프롬프트의 3요소

1. Persona (역할)

👉 "누구처럼 말할 것인가"

2. Context (상황)

👉 "어떤 배경에서 말하는가"

3. Task (할 일)

👉 "무엇을 하라는 것인가"

- 구조 공식

- [Persona] + [Context] + [Task]

AI 프롬프트 구조 설계

● Persona (역할)

• Persona란?

- AI에게 전문성과 말투를 부여

• ✕ 없음

- 설명해줘

• ✔ 있음

- "너는 10년차 마케터야"

✦ 효과

- 더 전문적
- 더 현실적
- 더 구체적

• 문장 개선하기

- "다이어트 방법 알려줘"

🗨️ Persona 추가

🗨️ 예시 답

"너는 전문 영양사야. 건강한 다이어트 방법을 설명해줘."

AI 프롬프트 구조 설계

● Context (맥락)

- Context 란?
 - AI가 참고해야 할 배경 정보
- ✕ 없음
 - 자기소개서 써줘
- ✔ 있음
 - "신입 개발자 지원용 자기소개서를 써줘. Python 프로젝트 경험 있음."

✦ 효과

- 정확도 상승
- 맞춤형 답변
- 불필요한 내용 감소

• 문장 개선하기

- "여행 추천해줘"

📄 Context 추가

📄 예시 답

"여름에 혼자 가는 3일 일본 여행 추천해줘. 예산 100만원."

AI 프롬프트 구조 설계

● Task (할 일)

• Task 란?

- AI에게 구체적인 행동 지시

• ✕ 모호

- 설명해줘

• ✔ 명확

- 3가지로 요약하고, 표로 정리해줘

✦ Task 구성 요소

- 형식 (리스트, 표)
- 길이 (3개, 5줄)
- 스타일 (쉽게, 전문가처럼)

• 문장 개선하기

- “AI 설명해줘”

🔗 Task 구체화

🔗 예시 답

"AI 개념을 초등학생도 이해할 수 있게 3줄로 설명해줘."

AI 프롬프트 구조 설계

- 완성된 프롬프트

- “너는 IT 강사야. AI 개념을 모르는 직장인을 대상으로, 쉽게 이해할 수 있도록 3가지 핵심 포인트로 설명해줘.”

- 업무 활용 예시

- 이메일 작성
- 보고서 생성
- 마케팅 카피
- 코드 생성

- 예시(업무)

- “너는 HR 매니저야. 신입 합격 안내 이메일을 정중하게 작성해줘.”
- “너는 10년차 콘텐츠 마케터야. IT 스타트업 블로그 글을 작성해야 해. SEO를 고려해서 5개의 소제목과 함께 글을 작성해줘.”

AI 프롬프트 구조 설계

- (고급 팁) 프롬프트 잘 만드는 3가지 원칙

1. 구체적으로 써라
2. 조건을 넣어라
3. 결과 형태를 지정하라

- 흔한 실수

- 너무 짧다
- 조건이 없다
- 역할이 없다

- (고급 팁) 템플릿 제공

너는 [Persona]야.

[Context] 상황에서,

[Task]를 수행해줘.

결과는 [형식]으로 작성해줘.

- 핵심 요약

- Persona = 역할
- Context = 상황
- Task = 지시

☞ 이 3개가 결과를 결정한다.

할루시네이션 방지

프롬프트 작성

1. 할루시네이션이 생기는 이유

- AI는 기본적으로

- “정답 생성기”가 아니라
- “그럴듯한 문장 생성기”임

- 그래서:

- 정보가 부족하면 → 채워 넣고
- 애매하면 → 일반화하고
- 모르면 → 추측함

- 결론

- 제어하지 않으면 반드시 만들어 냄

2. 가장 중요한 원칙(핵심 3가지)

1. “모르면 말하지 마라”를 강제해야 함

✗ 잘못된 프롬프트

- 최대한 자세히 설명해줘

✓ 개선

- 모르는 내용은 추측하지 말고 "모르겠습니다"라고 답해줘

2. 근거 기반 답변을 요구해야 함

✗ 잘못된 프롬프트

- 설명해줘

✓ 개선

- 근거와 함께 설명하고, 근거가 불확실하면 명시해줘

3. 범위를 제한해야 함

✗ 잘못된 프롬프트

- AI 역사 설명해줘

✓ 개선

- 2020년 이후 주요 변화만 설명해줘

3. 할루시네이션 방지 핵심 구조

- 실무에서 가장 많이 쓰는 구조:

너는 정확성을 최우선으로 하는 전문가야.

다음 규칙을 반드시 지켜:

1. 모르는 내용은 절대 추측하지 말고 "모르겠습니다"라고 답할 것
2. 불확실한 정보는 "불확실함"이라고 표시할 것
3. 근거가 있는 내용만 답변할 것
4. 가능하면 출처 또는 근거를 함께 제시할 것

[질문]

☞ 이 구조만으로도 정확도가 크게 올라감

4. 단계별 방지 기법

1단계: 추측 차단

- 추측하지 말고, 확실한 정보만 답해줘.

📌 효과: 기본적인 오류 감소

2단계: 불확실성 표시

- 확실한 정보와 불확실한 정보를 구분해서 표시해줘.

3단계: 근거 강제

- 각 주장마다 근거를 함께 제시해줘.

4단계: 검증 요구

- 답변을 작성한 후, 스스로 검토해서 틀릴 가능성이 있는 부분을 표시해줘.

5. 고급 프롬프트 (실무용)

- 버전 1: 안전 답변 모드

너는 매우 신중한 전문가야.

답변 규칙:

- 모르면 절대 추측하지 말 것
- 확실하지 않으면 "불확실"이라고 표시
- 근거 없는 내용은 작성 금지
- 필요한 경우 추가 정보 요청

질문:

5. 고급 프롬프트 (실무용)

- 버전 2: 검증 포함

다음 절차로 답변해:

1. 질문 이해
2. 알고 있는 사실만 정리
3. 불확실한 부분 구분
4. 최종 답변 작성
5. 오류 가능성 검토

질문:

5. 고급 프롬프트 (실무용)

- 버전 3: 업무용 (강력 추천)

너는 데이터 기반 의사결정을 하는 전문가야.

다음 규칙을 지켜:

1. 근거 없는 내용은 작성하지 마라
2. 추측이 필요한 경우 반드시 "추정"이라고 표시해라
3. 사실과 의견을 구분해라
4. 불확실한 정보는 제외하거나 명시해라

결과는 명확하고 검증 가능한 형태로 작성해라.

질문:

6. 가장 많이 하는 실수

- 실수 1 : “자세히 설명해줘” => 오히려 할루시네이션 증가
- 실수 2 : 범위 없음 => AI가 채워 넣음
- 실수 3 : 검증 없음 => 틀려도 그대로 출력됨

- 가장 강력한 한 줄

- 모르는 내용은 절대 추측하지 말고 "모르겠습니다"라고 답해줘.

- 단순하지만 효과 매우 큼

7. 완성형 템플릿(추천)

너는 정확성을 최우선으로 하는 전문가야.

다음 규칙을 반드시 지켜:

- 모르면 "모르겠습니다"라고 답할 것
- 추측 금지
- 불확실한 정보는 명시
- 근거 기반으로만 답변

질문:

8. 한 단계 더 UPGRADE

- 질문을 이렇게 바꾸면 더 좋아짐

- **답변 전에, 이 질문에 대해 확실히 알고 있는 것과 모르는 것을 먼저 구분해줘.**

☞ 이유

- AI가 “모르는 영역”을 먼저 인식함

9. 핵심 요약

- AI는 기본적으로 추측한다
- 따라서 반드시:
 - 추측 금지
 - 근거 요구
 - 불확실성 표시

☞ 이 3개를 강제해야 한다

업무용 프롬프트

템플릿 설계

1. 템플릿의 필요성

- 목표 : “왜 매번 새로 쓰면 안되는지” 이해
- “AI 쓸 때 매번 여러분은 다음과 같이 하시죠?”
 - 질문 다시 씀
 - 결과 들쭉날쭉
 - 품질 불안정
- 핵심 메시지
 - “AI는 잘 쓰는 사람이 아니라, 잘 ‘설계’하는 사람이 이깁니다.”
- Before / After
 - ✗ 일반 프롬프트
 - 매번 다름
 - ✓ 템플릿
 - 재사용 가능 / 품질 안정

* 핵심 개념

템플릿 = 재사용 가능한 프롬프트 구조

2. 템플릿 구조 설계

● 핵심 구조

[Persona]

[Context]

[Task]

[Constraint]

[Output Format]

● 요소별 설명

- [Persona]
 - 역할 (마케터, 기획자, 개발자)
- [Context]
 - 상황 (회사, 프로젝트, 대상)
- [Task]
 - 할 일 (작성, 분석, 요약)
- [Constraint]
 - 길이
 - 톤
 - 금지사항
 - 할루시네이션 방지
- [Output Format]
 - 표 / 리스트 / 문단
 - 바로 복붙 가능 형태

2. 템플릿 구조 설계

● 예시(업무용)

너는 B2B 마케팅 전문가야.

우리 회사는 SaaS 스타트업이고,
신규 고객 유입을 늘려야 해.

블로그 글 주제 5개를 제안해줘.

조건:

- SEO 키워드 포함
- 실무자가 관심 가질 주제
- 너무 일반적인 내용 금지

결과는 표로 작성해줘.

3. (실습)프롬프트 템플릿 설계해 보기

- Step 1 : 업무 정의(현재 직장에서)

- 반복되는 업무는?
- 시간이 많이 드는 작업은?

➤ 예시)

- 이메일 작성 / 보고서 요약 / SNS 콘텐츠

- Step 2 : 초안 만들기

- 템플릿 작성

너는 [Persona]야.

[Context]

[Task]

조건:

-

-

-

결과는 [형식]으로 작성해줘.

3. (실습)프롬프트 템플릿 설계해 보기

- Step 3 : 1차 실행 & 문제 찾기

- 직접 AI에 넣어 보기
- 체크 포인트:
 - 애매한 부분?
 - 결과 품질?
 - 빠진 조건?

- Step 4 : 개선

- 코칭 포인트
- 개선 방법
 - Persona 구체화
 - Context 추가
 - Constraint 강화
 - Output 명확화

4. 고도화

- 고급 기법 1 : 할루시네이션 방지

- 모르는 내용은 추측하지 말 것
- 불확실한 정보는 표시할 것

- 고급 기법 2 : 단계적 사고

- 1단계: 분석
- 2단계: 아이디어 생성
- 3단계: 최종 결과

- 고급 기법 3 : Few-shot

- 예시를 함께 제공

5. 최종 실무 활용 템플릿(배포용) 예시

● 이메일 템플릿

너는 회사의 커뮤니케이션 전문가야.

상황:

[상황 입력]

목표:

[목표]

이메일을 작성해줘.

조건:

- 정중한 톤
- 간결하게
- 핵심 전달

형식:

제목 + 본문

5. 최종 실무 활용 템플릿(배포용) 예시

● 보고서 요약

너는 데이터 분석가야.

다음 보고서를 요약해줘.

조건:

- 핵심 3가지
- 인사이트 포함
- 불필요 내용 제거

형식:

- 요약
- 인사이트

5. 최종 실무 활용 템플릿(배포용) 예시

● 콘텐츠 생성

너는 콘텐츠 마케터야.

[주제]

블로그 글을 작성해줘.

조건:

- SEO 고려
- 소제목 포함
- 실무 중심

형식:

제목 + 소제목 + 본문

도메인별 텍스트 정제 및 토큰화 실무 설계

* “데이터 전처리가 AI 성능을 결정한다”

1. 왜 전처리가 중요한가?

- 문제 인식

"AI 성능은 모델이 아니라 데이터가 결정한다"



"같은 모델을 써도 어떤 팀은 성능이 좋고, 어떤 팀은 안 좋습니다."
"차이는 대부분 '데이터 전처리'에서 나옵니다."

✕ 문제 사례

- 이상한 요약 결과
- 엉뚱한 키워드
- 검색 정확도 낮음

👉 원인

정제 부족
토큰화 오류
도메인 무시

👉 핵심 메시지

"Garbage in, Garbage out"

2. 텍스트 정제 설계

- 텍스트 정제란?

- 데이터를 “의미 있게” 만드는 과정

- 정제 3단계

1. 노이즈 제거 : URL, 특수문자
2. 정규화 : 대소문자, 반복 문자
3. 의미 보존 : 가장 중요

- 잘못된 정제

- 전부 소문자 변환
- 불용어 무조건 제거

- 좋은 정제

- 목적 기반
- 도메인 기반
- 최소 변경

3. 토큰화 전략

- 토큰화란?

- 텍스트를 모델 입력 단위로 분해

- 토큰화 방식

- 단어 기반
- 서브워드 (BPE, WordPiece)
- 문자 기반

- 왜 중요한가?

- 모델이 “이해하는 단위”가 됨

- 잘못된 토큰화

- 의미 단절
- OOV 증가

- 좋은 토큰화

- 의미 유지
- 희귀 단어 대응

4. 도메인별 설계

● 핵심 질문

- “이 데이터는 어떤 도메인인가?”

1. SNS 데이터

- 특징
 - 비정형
 - 오타 많음
 - 이모지
- 정제
 - 이모지 처리
 - 반복 문자 축소
 - URL 제거
- 토큰화
 - Subword 필수

2. 법률

- 특징
 - 정확성 중요
 - 긴 문장
- 정제
- 원문 유지
- 구조 유지
- 토큰화
 - 문장 단위 + 단어

3. 의료

- 특징
 - 약어 많음
 - 전문 용어
- 정제
 - 약어 확장
 - 단위 통일
- 토큰화
 - 도메인 vocab 필요

4. 도메인별 설계

- 핵심 질문

- “이 데이터는 어떤 도메인인가?”

4. 코드

- 특징
 - 기호 중요
 - 구조 중요
- 정제
 - 공백 유지
 - 특수문자 유지
- 토큰화
 - AST or Subword

5. 이커머스

- 특징
 - 짧은 텍스트
 - 키워드 중심
- 정제
 - 브랜드 통일
 - 단위 정규화
- 토큰화
 - 키워드 중심

5. 실무 적용

- 실무 설계 원칙

1. 도메인 먼저 정의
2. 의미 보존 우선
3. 모델 목적 기준

- 핵심 요약

- 정제 = 품질
- 토큰화 = 구조
- 도메인이 모든 것을 결정
- 정답은 없다 → 설계가 중요
- 도메인 이해가 핵심
- 과도한 정제 금지